

Adapt Burstable Containers to Variable CPU Resources

Hang Huang, Yuqing Zhao, Jia Rao[✉], *Member, IEEE*, Song Wu[✉], *Member, IEEE*, Hai Jin[✉], *Fellow, IEEE*, Duoqiang Wang[✉], Suo Kun[✉], *Member, IEEE*, and Lisong Pan

Abstract—In the age of the cloud-native, container technology, referred as OS-level virtualization, is increasingly adopted to deploy cloud applications. Compared with virtual machines, containers are lightweight and flexible in resource management. An important quality-of-service (QoS) class in container management is burstable container, whose resource limits are higher than the actual requests allowing a container to expand whenever demands ramp up and additional resources become available. However, efficiently managing burstable containers is challenging, especially for CPU resources. On the one hand, burstable containers should maintain sufficient concurrency, in the form of threads, to utilize extendable CPU resources. On the other hand, the degree of concurrency necessary for utilizing peak CPU resources leads to suboptimal performance when a container's CPU allocation is constrained. In this paper, we recommend that the number of threads in burstable containers should always be set to the CPU limit to guarantee extensibility. However, modern operating systems (OSes) fall short of efficiently managing thread oversubscription. First, the OS CPU scheduler is inefficient for scheduling excessive threads and lacks container awareness. Second, the existing blocking synchronization supported by the OS kernel is inefficient in handling the sleep and wakeup of excessive threads. Finally, the non-blocking synchronization may waste CPUs performing busy waiting when more than one thread in the run queue. To this end, we present a user-level adaptive container scheduler and two OS mechanisms, *virtual blocking* and *busy-waiting detection*, to avoid inefficiency in managing burstable containers without requiring program code changes. Experimental results show that our approaches can keep burstable containers efficient while allowing the applications in containers to take advantage of additional CPUs. The performance gain under high system load is up to $29.7\times$.

Index Terms—Container, oversubscription, scheduling

1 INTRODUCTION

WITH the increasing popularity of cloud-native, container technology has become widely used to build and deploy scalable applications in the cloud. Compared with virtual machines (VM), containers implement lightweight OS-level virtualization based on kernel sharing. By eliminating the virtual hardware abstraction, containers can achieve more elastic resource management. Therefore, users prefer running programs in separate containers and placing containers that run related programs on the same pod through the orchestration system. Programs running in the container can create multiple threads to perform tasks concurrently

and keep high resource utilization. The container employs the mechanism of request and limit to control resource allocation and specifies the resource request and limit based on its QoS class. Two commonly-used QoS classes in the cloud are guaranteed and burstable. The guaranteed container always sets the same value for its resource request and limit and never accesses excessive resources. In contrast, the burstable container always sets a higher resource limit than the request and can acquire additional idle resources under the premise of not exceeding the limit. Therefore, burstable containers play an important role in improving resource utilization in the cloud, as their available resources vary dynamically in response to their resource requests, limits, and system load.

However, with distinct settings and system loads, different resources might have various effects on burstable containers. The resources, in the case of different behavior, can be categorized into compressible resources and incompressible resources. If incompressible resources, such as memory and storage, are exhausted, the burstable container may be killed, and the server becomes unstable. Managing incompressible resources should be more complicated and cautious. For example, Pufferfish [1] presents an elastic memory manager for data-intensive containers with the consideration of OOM and requires the application equipped with dynamic memory allocation like JVM. In comparison, once the burstable container starts hitting the compressible resources (e.g., CPU) limit, it will be throttled rather than terminated or evicted. Therefore, the requests and limits of the incompressible resources are always carefully set to the same value to

- Hang Huang, Yuqing Zhao, Song Wu, Hai Jin, Duoqiang Wang, and Lisong Pan are with the National Engineering Research Center for Big Data Technology and System Services Computing Technology and System Lab, Cluster and Grid Computing Lab, Big Data Technology and System Lab, School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan 430074, China. E-mail: {huanghang, yuqingzhao, wusong, hjin, dqwang, catoi}@hust.edu.cn.
- Jia Rao is with the Department of Computer Science, University of Texas, Arlington, TX 76019 USA. E-mail: jia.rao@uta.edu.
- Suo Kun is with the Department of Computer Science, Kennesaw State University, Kennesaw, GA 30144 USA. E-mail: ksuo@kennesaw.edu.

Manuscript received 24 June 2021; revised 3 January 2022; accepted 6 March 2022. Date of publication 12 May 2022; date of current version 10 February 2023.

This work was supported by the National Science Foundation of China under Grant 62032008.

(Corresponding authors: Jia Rao and Song Wu.)

Recommended for acceptance by J. Chen.

Digital Object Identifier no. 10.1109/TC.2022.3174480

prevent accidental killing, while the CPU resources are often-times set to different values to improve flexibility and utilization.

While elastic CPU resource management of burstable containers helps attain higher system utilization, forcing modern applications to adapt to variable CPU resources is still challenging. Applications that only support constant CPU resource allocation cannot fully exploit varying CPU resources and could perform even worse due to misbehavior with CPU resources. Generally, whether the CPU resource is effectively used is tightly coupled with the application parallelism. The application should create threads the same as the number of CPUs to effectively utilize the available CPU resources. However, when the global CPU resources are scarce, and the container can only obtain the limited CPUs, competition on fewer CPUs among large amounts of threads would contribute to severe performance degradation.

Existing researches focus on tuning an appropriate degree of parallelism based on workload characteristics, shared resource competition as well as system load. For instance, application parallelism management has been designed and optimized in compilers [2], user-level thread library [3], and runtime systems [4]. However, these approaches require users to rebuild their container images to contain modified applications or libraries. Other approaches implement dynamic threading algorithms to alter the number of working threads in a thread pool. For example, OpenMP [5] determines the number of working threads before starting each parallel execution. However, as the container lacks isolated system resource statistics [6], the algorithms may even negatively affect application performance in the container. Other studies, including offline statistical learning [7], [8], [9] and online runtime adaptation [10], [11], [12], target in optimizing thread scheduling to improve locality and reduce competition. However, these studies do not consider the correlation between different containers. Normally, relevant containers should be scheduled as close as possible.

In this paper, we focus on burstable containers, in which the number of threads is the same as the CPU limit instead of considering the variable number of real-time available CPUs. Notably, it is meaningless to set the CPU limit more than the maximum number of CPUs the application can scale up to. Provisioning enough threads is to allow the application to utilize idle CPU resources as the available CPUs varies. Here the main challenge is how to co-schedule multiple burstable containers and manage over-provisioned threads on fewer CPUs efficiently. Ideally, compared to provisioning as many threads as its CPU request, a container provisioning as many threads as its CPU limit should perform similarly when the system load is high and adapt to additional CPUs when the system load is low. However, we observed that the burstable containers experienced severe performance degradation under high system load.

To study the characteristics of running multi-threaded applications with different parallelism in containers, we started burstable containers that have a request of 8 CPUs and a limit of 32 CPUs to run benchmarks from NPB [13] with two configurations: (1) 8 threads, the same as the guaranteed CPUs; (2) 32 threads, the same as the most obtainable CPUs. We simulated various system loads by

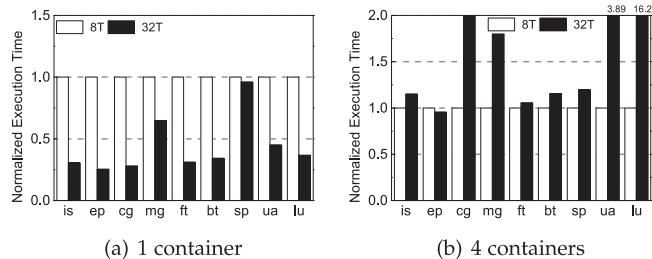


Fig. 1. Performance of burstable containers (a request of 8 CPUs and a limit of 32 CPUs) with different degree of parallelism and system loads.

running a different number of containers simultaneously. As shown in Fig. 1a, when the system executes a single container, the container with 32 threads can effectively utilize the unused CPU resources and brings up to 4× performance improvement compared with the container with only 8 threads. The above results show that determining the degree of parallelism based on CPU limit helps enhance higher CPU utilization and achieve better performance. In comparison, when the system is under full load with four burstable containers, each one can only obtain the requested CPUs. At this time, each container cannot obtain additional CPUs, and excessive threads can cause severe competition inside the container and among containers. Fig. 1b depicts that the performance loss can reach up to 94%. Therefore, simply determining the degree of parallelism based on CPU limit is inefficient under high system load.

This paper presents a thorough investigation of the overhead and inefficiencies in burstable containers equipped with excessive threads (Section 3). We found that the current kernel scheduling mechanisms are the culprit of competition among containers. First, the kernel CPU scheduler manages containers only considering the CPU request and limit while lacking container awareness. As a result, threads of the same containers are randomly scheduled among all CPUs, which leads to frequent thread migrations and severe competition between distinct containers. Second, due to random threads placement and frequent migrations, the data placement optimization algorithm of the kernel scheduler becomes exhausted and introduces dramatic overhead. Furthermore, we also revealed two previously understudied issues in threads synchronization, which are the culprits of significant performance slowdowns when various threads are running in fewer CPUs. First, the existing mechanisms in blocking synchronization are inefficient to wake up and sleep excessive threads and induce unnecessary threads migrations. Second, threads with busy-waiting (spinning) synchronization might waste CPUs and delay the execution of threads residing on the same CPU.

Based on these findings, we believe the overhead of excessive threads from the burstable container can be effectively avoided by optimizing scheduling and synchronization. To this end, we develop a user-level container scheduler and two synchronization mechanisms to manage burstable containers well. *Adaptive container scheduler* (Section 4.1) is a new mechanism for container scheduling that assigns dedicated CPUs to containers with the consideration of multiple dimensions. It makes container scheduling decisions based on hardware architecture information, container relationship, and system loads. *Virtual blocking* (Section 4.2) is a new mechanism for

futex-based blocking synchronization that enables efficient thread sleep/wakeup. It emulates the effect of blocking through CPU run queue operations without putting threads to actual sleep. *Busy-waiting detection* (Section 4.3) is a new mechanism for preventing useless busy-waiting operations. It is valid for any busy-waiting implementations. Experimental results (Section 5) on both micro- and macro-benchmarks show that our solutions can significantly improve the performance of burstable containers under high system loads.

2 BACKGROUND AND MOTIVATION

2.1 Container-Based Virtualization

Container-based virtualization, also called operating-system level virtualization, is an approach to provide multiple virtual execution environments for applications on top of the host operating system. Containers support packaging up an application with all its parts in an isolated environment without any dependencies on the hosts. Compared with VMs, containers are exceptionally lightweight as they do not contain complete guest operating systems and virtual hardware abstractions. Instead, to achieve isolation and security, containers only include essential libraries and layered root file systems, which rely on several kernel mechanisms such as namespaces and cgroups.

The namespace, which defines the boundaries of the awareness of execution environments for processes, is a fundamental component of containers. The Linux kernel currently supports seven namespaces (*USER*, *UTS*, *NET*, *PID*, *MNT*, *IPC*, and *CGROUP*) to grant restricted host system views for containers. For example, the *USER* namespace isolates user and group identifiers for containers, allowing users to own dedicated user IDs and group IDs. Meanwhile, a namespace can be shared among relevant containers. A group of relevant containers serving for the same service always share *NET*, *PID*, and *IPC* namespaces. Those containers run like on a logical host and can communicate using “localhost” or standard inter-process communications.

Control groups or cgroups are kernel mechanisms that provide a unified interface to limit, account, and isolate resource usage for processes in the container. Processes in the same container are together bound to the same set of limits. The Linux kernel already supports 13 cgroups subsystems, such as *CPUACCT*, *CPUSET*, etc. A cgroup manages resource by resource requesting and limiting, guaranteeing the container being assigned requested resources and restricting the most resources it can acquire. However, various resources might perform differently when the limit is more than the request. For incompressible resources, such as memory and disk, which are stateful, allocated and reclaimed slowly, the request and the limit are often carefully set to the same value to avoid unpredictable errors caused by resource shortage. In comparison, the compressible and stateless resources, such as CPU, the request and the limit are sometimes set to different values to improve system CPU usages because the shortage of CPUs does not lead to the misbehavior of containers.

2.2 Burstable Container

The container whose resource limit is higher than the request is known as a burstable container. In this paper, we focus on burstable containers whose resource limits and

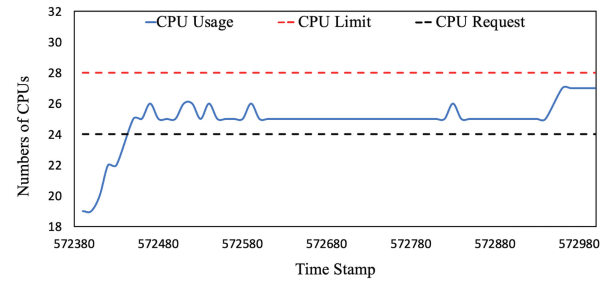


Fig. 2. The CPU usage of a overcommitted container.

requests are different only for CPU resources. To manage CPU resources of containers, cgroups provide the CPU shares to define their CPU requests. The amount of CPU time allocated to a given container refers to the number of shares divided by the total amount available. Cgroups implement CPU limits for containers by CPU quota and CPU period. A container can execute for up to *cpu.cfs_quota_us* microseconds during a period of *cpu.cfs_period_us* microseconds. Generally, burstable containers can consume more available CPU resources exceeding the CPU requests but not exceeding the CPU limits.

To understand the adoption of burstable containers in industries, we analyze the open cluster trace of 4000 machines in a period of 8 days presented by Alibaba [14]. Notably, the resource requests and limits of containers remain unchanged during execution time. First, as we mentioned earlier, the request and limit of those incompressible resources in all containers are set to the same value. Second, we found 7.66% of all 370,540 containers are burstable containers, and 2.78% of them have a $2\times$ higher CPU limit than the request. Third, during lifecycles, burstable containers can always gain more CPUs than their request because of the low load of the cluster. For example, as shown in Fig. 2, a burstable container with a request of 24 CPUs and a limit of 28 CPUs executes on a host equipped with 96 CPUs. As the total CPU utilization of the system is always below 100%, the burstable container consumes more than 24 CPUs (requested CPUs) during most of its lifecycles and improves host resource utilization compared to only consuming 24 requested CPUs.

2.3 The Impact of Oversubscribed Threads

According to the previous discussion, there are also many guaranteed containers whose requests and limits are the same in the cloud. In realistic workloads, there exists a mixture of burstable containers and guaranteed containers together. It is necessary to create burstable containers to utilize extra accessible CPUs when more CPU resources become idle. It is pretty common as the utilization of the cloud varies throughout the day. Meanwhile, this might introduce thread contention and performance degradation when the global CPU resources of the host are under pressure. In this section, we discuss the potential impact of the burstable container with a different number of threads.

We started four containers on a 32-CPU machine running NPB benchmarks to make system CPUs under pressure. The three are guaranteed containers with a configuration of the same request and limit of 8 CPUs, while the rest is a burstable container with the request of 8 CPUs and the limit of

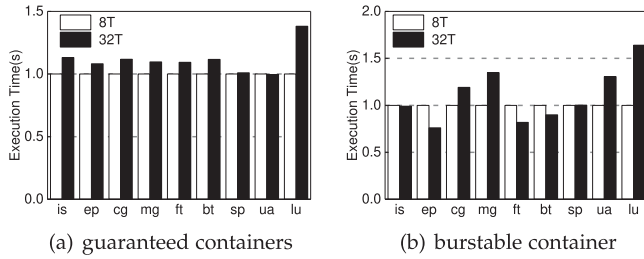


Fig. 3. The performance of co-locating four containers.

32 CPUs. The guaranteed containers always run benchmarks with 8 threads. The burstable container runs benchmarks with two settings: (1) 8 threads, which is equal to the number of requested CPUs; (2) 32 threads, which is the same as the number of limited CPUs. Figs. 3a and 3b show the results normalized with the burstable container running 8 threads. From the above results, we first observe that the average performance of three guaranteed containers drops when the burstable container creates more threads. Also, the performance of the burstable container increases in some benchmarks (e.g., ep, ft, bt) at the expense of guaranteed containers' performance. In addition, we find that the performance of the burstable container still drops seriously in some benchmarks (e.g., cg, mg, ua, lu). All the experiments show that burstable containers with excessive threads have a significant impact on the performance of themselves as well as other containers. Ideally, burstable containers are expected to improve performance under low load without degrading the overall performance under high load. In the next section, we explore and discuss the root causes of the performance loss.

3 ROOT OVERHEAD ANALYSIS

3.1 Inefficiencies in Scheduling Oversubscribed Threads

The Linux default scheduler, known as Completely Fair Scheduler (CFS), implements a specific form of fair scheduling to ensure fairness among all tasks. Each thread on the CPU run queue is guaranteed to obtain an amount of CPU time based on its relative share. Once the thread consumes up all its timeslices, it will be put back into the CPU run queue to wait until the next scheduling period. To optimize global CPU utilization and memory efficiency, the CFS proposes load balancing and NUMA balancing mechanisms. However, when the number of threads exceeds the number of CPUs, those mechanisms will not be effective and even negatively impact application performance. As the designs adopt only the global optimal strategy, the following discussions show that they might not perform well for a single container.

Load Balancing. CPU load refers to the total loads of threads on a CPU and is used to reflect the busyness of the CPU. CFS implements load balancing to balance the loads among CPUs and evenly distribute the threads across CPUs. When one thread is newly created or awakened, CFS first selects the idle CPU and inserts it into the corresponding CPU run queue. Furthermore, the load balancing is also triggered in the following situations. First, when a CPU becomes idle, CFS starts load balancing to pull threads from

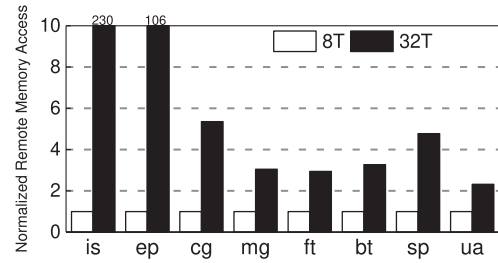


Fig. 4. The remote memory access of co-locating four containers.

other busy CPUs to the idle CPU. Second, CFS periodically checks the load of all CPUs and starts load balancing once the load is imbalanced.

The load balancing helps avoid the situation where many threads compete under a small number of CPUs. However, when there exist oversubscribed threads, the existing load balancing mechanism might be ineffective: 1) It is difficult to guarantee that the awakened thread is still placed on the previously running CPU, which results in a loss of locality. 2) The threads of the same container are randomly scheduled to the CPUs on different NUMA nodes, leading to high communication delay. 3) Threads of different containers could be scheduled on the same CPU, affecting cache efficiency seriously.

NUMA Balancing. The kernel also implements NUMA balancing to improve the performance of applications on NUMA machines. Generally, accessing local memory is much faster than accessing remote memory. An application performs better if all threads are scheduled on the same node. The goal of NUMA balancing is to make threads and their memory objects as close as possible. Specifically, it periodically unmaps parts of thread memory and tracks their location based on NUMA hinting fault. Once a thread accesses the memory twice from a remote NUMA node, NUMA balancing moves threads closer to the memory they need to access or move memory closer to the related thread. However, when the system suffers from thread oversubscription, keeping memory closer to threads is challenging due to lacking container awareness. First, as memory objects are shared between threads that are scheduled evenly across all nodes, NUMA balancing cannot easily ensure threads and their memory on the same node. Second, as load balancing migrates threads frequently, it is difficult and expensive for NUMA balancing to migrate memory close to related threads. As shown in Fig. 4, co-scheduling four burstable containers with 32 threads on 32 CPUs introduces at least twice remote memory access compared with only 8 threads.

The most straightforward way to optimize threads and data placement is to bind a container with fixed CPUs, avoiding frequent thread and data migrations and improving locality. However, the binding method might waste idle CPUs and lead to low CPU utilization. Meanwhile, it is difficult for users to customize the binding policy as they do not know which machine the container will be scheduled and the CPU topology of that machine in the cluster. Some other work [15] focuses on coscheduling multiple applications on the same machine or introducing new runtimes to dynamically adjust the degree of parallelism based on the system load. However, these approaches have not been proven

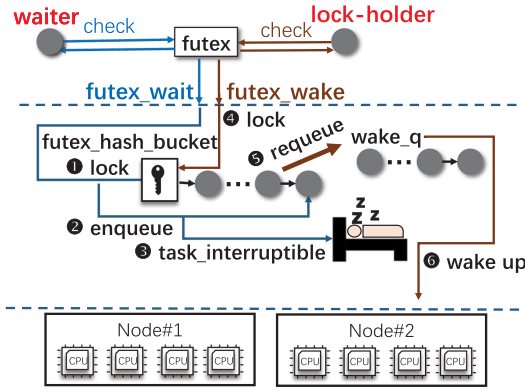


Fig. 5. The blocking synchronization with *futex*.

effective in containers because applications in containers are hard to acquire the local resources statistics. Moreover, application modifications are required in those systems, which are not friendly to containers. Other solutions [9], [16] leverage the learning model to study the workload in advance and get the optimized scheduling policy. However, it is challenging to apply these techniques to container environments, which are incredibly complex, diverse, and dynamic.

3.2 Inefficiencies in Synchronizing Oversubscribed Threads

Blocking Synchronization. In blocking synchronization, when a caller thread fails to acquire the lock or cannot meet a certain condition, it is dequeued from the CPU run queue to wait idly on the sleep queue. The thread is later woken up when the lock becomes available or the condition is met. Compared to busy-waiting synchronization, which repeatedly tries to obtain a lock or verifies a condition, blocking synchronization saves many CPU cycles for unsuccessful synchronization through trapping into the kernel to sleep. In Linux, the kernel offers various mechanisms to handle thread sleep and wakeup for blocking synchronization. Without loss of generality, we use the design of *futex* to illustrate how Linux handles thread sleep and wakeup (as shown in Fig. 5).

Fast userspace mutex (*futex*) is a blocking synchronization mechanism supported by the OS kernel. It is widely used in traditional multi-threaded applications. The *futex* is in essence a userspace variable. When the value of *futex* indicates no contention, the successful synchronization entirely completes in userspace. However, when the value of *futex* indicates contentions, the unsuccessful synchronization traps into the kernel to process thread sleep through steps depicted in Fig. 5. The waiter thread first invokes *futex_wait* to trap into the kernel. After double-checking the userspace variable, it acquires the lock that protects the *futex_hash_bucket* queue (step ①) and then enqueued to the tail of sleep queue (step ②). After that, the runtime state of the waiter changes from “runnable” to “sleep” (*TASK_INTERRUPTIBLE*), and the CPU scheduler removes the waiter from the CPU run queue (step ③). When the lock holder releases the userspace lock, it also needs to invoke *futex_wake* to wake up sleeping threads. It acquires the lock on the *futex_hash_bucket* (step ④) and migrates

TABLE 1
Runtime Statistics of Different Number of Threads

Apps # of threads	ua		lu	
	8 T	32 T	8 T	32 T
Execution time(s)	113.37	157.81	109.69	2739.24
CPU utilization(%)	777	618	791.5	798.8
# of migrations	47	392285	35	28314
# of instructions (B)	2540.34	2538.3	3375.2	8714.7

waiter(s) in the sleep queue to a temporary wakeup queue (*wake_q*) (step ⑤), from where the waiter(s) awoken (step ⑥). The design purpose is to prevent holding the lock of the *futex_hash_bucket* for a long time as thread wakeup operations may consume much time. At last, the lock holder performs actually wakeup operations and wakes up waiters one at a time.

Next, we reveal why the sleep and wakeup process is expensive when threads are oversubscribed. First, the existing wakeup process is too complicated. Although thread sleep is not inside the critical section, the wakeup process still greatly delays program execution. The wakeup operation (step ⑥ in Fig. 5) should select the idlest CPU and insert the awakening thread into its run queue. This operation requires locking the run queue of the new CPU. After that, the kernel checks preemption on the newly selected CPU. When the number of awakening threads is far more than CPUs, run queue locking and possible preemption can cause serious serialization and cascading performance degradation. Furthermore, moving waiters to the temporary *wake_q* queue (step ⑤ in Fig. 5) adds more serialization. Second, frequent thread sleep and wakeup may trigger excessive, unnecessary migrations. Thread sleep and wakeup involve state transitions between “runnable” and “sleep,” while only runnable threads contribute to the active load on a CPU. Linux performs thread migrations immediately to balance load once a significant imbalance is detected. When many threads perform blocking synchronization, frequent state switching and fluctuating CPU load will happen, leading to many more migrations. To quantify the inefficiency, we start different numbers of threads running a blocking-intensive benchmark (UA) on an 8-CPU container. As shown in Table 1, running 4x threads brings 38% overhead. The number of retired instructions only changes slightly, and the overhead is mainly due to much CPU time wasted on the run queue operations and surge of thread migrations.

Busy-Waiting Synchronization. In busy-waiting synchronization, threads could enjoy fast lock acquisition at the cost of wasting CPU cycles on busy-waiting. The wasted CPU time has little impact on the overall program performance if each thread monopolizes a CPU because the CPU would otherwise be idle. However, if threads are oversubscribed and multiple threads are placed on the same CPU, busy-waiting can burn the CPU time that other threads can use to do valuable work. Oversubscription also exacerbates the *lock-holder preemption* (LHP) problem, as the waiter thread might exhaust its time slices to busy-wait and become a lock holder that will soon be preempted. To identify the impact of busy-waiting, we start different numbers of

threads running a busy-waiting-based benchmark (LU) on an 8-CPU container. As shown in Table 1, although starting 32 threads can maintain high CPU utilization, it still suffers a 24x increase in execution time. The number of retired instructions is sharp, indicating that busy-waiting brings many more useless instructions and wastes much CPU time.

It is necessary to stop busy-waiting threads when there exist other runnable threads on the CPU. Towards this goal, many software or hardware-based methods have been developed. For instance, some software approaches [17], [18] present the spin-then-block strategy and set a threshold of the applied CPU cycles for each busy-waiting loop. However, these approaches require application modifications and are inefficient when the system is under-subscribed. Hardware approaches, e.g., Intel Pause-Loop-Exiting (PLE) [19] and AMD Pause Filter (PF) [20], detect and stop excessive busy-waiting based on hardware events. Specifically, PLE and PF monitor the frequency of the PAUSE or NOP instructions, the key instructions in most busy-waiting implementations. For example, the implementation of widely-used *pthread*s spin-lock (on the left) contains the NOP instruction in the busy-waiting loop. Regrettably, those hardware approaches are only valid for virtual CPUs (vCPUs) in virtualized environments. In addition, numerous spin implementations contain no special hardware instruction. For example, the busy-waiting implementation in *lu* is just a busy loop continuously checking a variable. PLE and PF are both invalid for this kind of spin implementation.

<pre>Pthread_spin_lock--glibc /** Program Code **/ do{ atomic_spin_nop(); val = atomic_load_relaxed (lock); } while (val!=0) /** Assembly Code **/ .L2: movl \$0, %eax call atomic_spin_nop call atomic_load_relaxed cmpl \$0, -20(%rbp) jne .L2</pre>	<pre>Sync_right--lu /** Program Code **/ do while (isync(iam).eq.1) !\$omp flush(isync) end do /** Assembly Code **/ .L10: cmpl \$1, %eax jne .L9 jmp .L10</pre>
---	---

Listing 1. Various busy-waiting implementations.

Summary. Our analysis has shown that the inefficiencies in kernel scheduling and synchronization are the root causes of the inefficiency in managing burstable containers. The kernel scheduler is inefficient in scheduling multiple burstable containers with excessive threads as the global optimizations fail due to the lack of container awareness. The synchronization mechanisms do not consider the scenario of multiple threads synchronizing on fewer CPUs and incur significant overhead. We believe that managing burstable containers based on efficient scheduling and synchronization can effectively eliminate these overheads.

4 DESIGN AND IMPLEMENTATION

In this section, we present a user-level mechanism to address the inefficiencies of container co-scheduling discussed in Section 3.1 and two synchronization mechanisms

to address the inefficiencies of synchronizing oversubscribed threads mentioned in Section 3.2.

4.1 Adaptive Container Scheduler

After the container orchestration system schedules a container to the most suitable server with enough resources, the container is placed on the selected server and scheduled based on the original kernel scheduler. The container can always obtain the requested CPU resources, and the total requested CPUs of all containers are always less than the total CPUs on the server. Adaptive container scheduler (ACS) is designed to help the original kernel scheduler co-schedule multiple containers flexibly and efficiently on the same server. It works as a daemon at the user level and consists of two components: a static CPUs allocator (SCA) and a dynamic CPUs adjustor (DCA). The static CPUs allocator assigns requested CPUs for containers at their startup based on their CPU resource configurations, the CPU topology, and the relationship with other containers, and reclaims allocated CPUs after they are destroyed. Dynamic CPUs adjustor periodically reorients the CPU allocation of each container based on their real-time CPU usage.

4.1.1 Static CPUs Allocator

The static CPUs allocator (SCA) monitors the startup and finish of containers at the user level and schedules containers to dedicated CPUs. It assigns each new container with requested CPUs, and the CPU allocation considers the following factors:

Algorithm 1. The Allocation of CPUs

- 1: **Variables:** CPU request of the i^{th} container req_i ; Idle CPU set $\mathbb{P}$; Container set $\mathbb{C}$; The most related container C_{relate} ;
 - 2: **Output:** The CPUs (C_i) allocated for container i .
 - 3: $req_i = \min(\frac{cpu.shares}{1024}, \frac{cpu.quota}{cpu.period})$
 - 4: /* Update container relationship */
 - 5: **for each** c in $\mathbb{C}$ **do**
 - 6: calculate relationship between c and the i^{th} container
 - 7: update C_{relate} of the i^{th} container;
 - 8: **end for**
 - 9: **if** $req_i > \mathbb{P}$ **then**
 - 10: reclaim overcommitted CPUs from $\mathbb{C}$
 - 11: **end if**
 - 12: $C_i \leftarrow$ top req_i closest CPUs of $\mathbb{P}$
 - 13: set the CPUSET cgroup of the i^{th} container
-

*CPU topology. Different CPUs may share different amounts of storage. The CPUs belonging to the same NUMA node share the local memory. The CPUs belonging to the same socket share the L3 cache. The CPUs that are logical CPUs of the same physical CPU share L1 and L2 cache. Therefore, the static CPUs allocator collects the system CPU information and calculates the CPU relationship based on the amounts of shared storage at startup. As shown below, we use C_{mn} to represent the relationship between CPU m and n

$$C_{mn} = 4 * Node_{mn} + 2 * Socket_{mn} + CPU_{mn}.$$

The higher the C_{mn} is, the more related the two CPUs are, as they share more storage. Here $Node_{mn}$, $Socket_{mn}$ and CPU_{mn} represent whether two CPUs are on the same NUMA node, socket, or physical CPU. Based on the amount of shared storage, we set different factors for each of them.

*CPU configurations. The CPU configurations of the container include cgroup parameters: $cpu.shares$, $cpu.quota$, and $cpu.period$. As most orchestration systems set $cpu.shares$ to 1024 to represent one CPU, the CPU request (C_{req}) and CPU limit (C_{limit}) of the container can be calculated based on the formula below:

$$C_{req} = \frac{cpu.shares}{1024}; C_{limit} = \frac{cpu.quota}{cpu.period}.$$

*Container relationship. We consider four factors to calculate the relationship (R_{ij}) between container i and container j . The mnt refers to the number of the same mount points between them. The ipc , pid , and net represent whether they have the same namespaces. We can calculate the R_{ij} using the formula below

$$R_{ij} = mnt_{ij} + ipc_{ij} + pid_{ij} + net_{ij}.$$

The larger R_{ij} is, the more related the two containers are. Every time a container is newly created or destroyed, the static CPUs allocator will update the global relationship map.

As shown in Algorithm 1, the static CPUs allocator adopts the best-effort algorithm to ensure that relevant containers are placed as close as possible. When it detects a newly created container, it calculates the container's CPU request and the relationship between the container and all other containers. Based on these, the static CPUs allocator finds the most related container and assigns idle CPUs close to the selected container to the newly created container. If the idle CPUs are less than the request, the static CPUs allocator reclaims CPUs from those burstable containers with overcommitted CPUs.

4.1.2 Dynamic CPUs Adjustor

Allocating different CPUs to each container can avoid competition between containers and unnecessary migrations. However, it also introduces problems such as low CPU utilization. Some containers might be unable to use up all assigned CPUs. The dynamic CPUs adjustor (DCA) periodically collects the load information of containers and adjusts the container's CPU allocation to react to the load changes and improve global CPU utilization. The period is set to 1 s by default and is allowed to adjust in response to different workloads. As the container cannot use more CPUs than its CPU limit, the dynamic CPUs adjustor never expands CPUs exceeding the CPU limit. Meanwhile, each container is assigned at least one CPU to ensure it can run normally and expand CPUs once under pressure.

Algorithm 2 shows the dynamic adjustment of allocated CPUs for each container. First, the DCA obtains the total CPU time (in nanoseconds) used by the container through its $cpuacct$ cgroup and calculates the actual used CPUs. Then the DCA makes a comparison between the container's

actual CPU demand and CPU allocation. The DCA uses heuristic algorithms to slowly adjust the container's CPUs to get close to the actual CPU demand. If at least one CPU is wasted, the DCA labels the container as requiring to shrink allocated CPUs. If no more than 0.05 CPUs are idle, the DCA marks the container as requiring to expand allocated CPUs, because the container is likely to consume up all allocated CPUs soon. If the container has been marked times more than the threshold that is by default set to five, the DCA begins to tune the allocated CPUs for the container through resetting the $CPUSET$ cgroup. If allocated CPUs of the container need to shrink, the DCA reclaims the farthest CPU among all allocated CPUs and turns it into an idle CPU. If the container requires more CPUs, the dynamic CPUs adjustor allocates one closest CPU of all idle CPUs. Once the allocated CPUs are adjusted successfully, the saved flags of the container will be cleared.

Algorithm 2. The Dynamic CPU Adoption

```

1: Variables: idle CPU set  $\mathbb{P}$ ; Container set  $\mathbb{C}$ ; The CPUs ( $C_i$ )
   allocated for container  $i$ ; The total CPU time of container  $i$ 
   used in current period  $T_{cur}$ ; The total CPU time of container
    $i$  used before current period  $T_{last}$ ; The number of shrink
   signal of the container  $shrink$ ; The number of expand sig-
   nal of the container  $expand$ ; The threshold of the signal
    $threshold$ ;
2: Output: The new allocated CPUs set ( $C_i$ ) for container  $i$ .
3: /* Update CPU set for all containers */
4: for each  $c$  in  $\mathbb{C}$  do
5:    $C_{used} = \frac{T_{cur} - T_{last}}{period}$ 
6:   if ( $C_i - C_{used}$ ) >= 1 then
7:      $shrink++$ 
8:   else
9:      $shrink = 0$ 
10:  end if
11:  if ( $C_i - C_{used}$ ) <= 0.05 then
12:     $expand++$ 
13:  else
14:     $expand = 0$ 
15:  end if
16:  if ( $shrink > threshold$ ) then
17:    reclaim one CPU to  $\mathbb{P}$ .
18:     $shrink = 0$ 
19:  end if
20:  if  $expand > threshold$  then
21:    allocate one CPU from  $\mathbb{P}$ .
22:    if SUCCESS then
23:       $expand = 0$ 
24:    end if
25:  end if
26: end for

```

Implementation. We implement the adaptive container scheduler (ACS) at the user level. To avoid dependence on container runtime, it monitors the creation and destruction of containers through utilizing the kernel feature inotify. We configure the inotify to monitor the filesystem events of the container cgroups. We implement dynamic two-dimensional linked lists to record the relationship between containers and the relationship between CPUs. The scheduling is truly reflected through adjusting the $CPUSET$ cgroup.

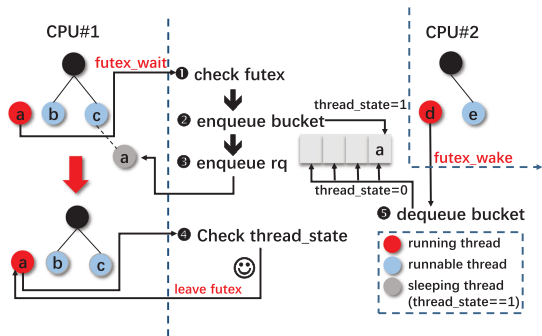


Fig. 6. Virtual blocking in futex.

4.2 Virtual Blocking

In the sleep and wakeup process, threads are moved between the sleep queue and the run queue. Those queue transitions trigger a large number of lock operations and thread state changes. When many threads of a burstable container run on fewer CPUs, these operations become very expensive. For instance, once multiple threads are awakened from the same CPU, the lock contention can be more intense, and the threads are more likely to be awakened in other CPUs instead of the previously running one. The essence of blocking is to disallow blocking threads from obtaining CPU cycles. Virtual blocking (VB) emulates the blocking by ignoring blocked threads when picking up the next running thread. Therefore, sleep queues disappear, thereby avoiding much of the overhead of thread wakeup.

VB determines whether the thread is blocked or not through the flag `thread_state` of the thread. Blocked threads, whose `thread_state` equals 1, are skipped when the CPU scheduler selects the next running thread until their `thread_state` becomes 0. Fig. 6 illustrates how VB works with futex. VB still retains the `futex_hash_bucket` queue in futex to maintain the order threads are blocked and waked up. However, VB never moves threads from `futex_hash_bucket` queue to the CPU run queue. When a thread is awakened by `futex_wake`, its `thread_state` is reset to 0 and it is removed from the `futex_hash_bucket` queue. VB then moves the blocked threads to the end of the CPU run queue. This design ensures that blocked threads will never have a chance to run, as long as there is at least one runnable thread with `thread_state` being 0 in the same CPU run queue. If all threads on the same CPU are blocked, which is uncommon in the case of thread oversubscription, each blocked thread takes a short turn run to check whether its `thread_state` has been cleared. After `thread_state` becomes 0, a thread resumes normal scheduling and is “awakened” from virtual blocking.

Since VB never puts threads into real sleep, an essential change in futex is necessary. Instead of relinquishing the CPU after a thread is enqueued to the `futex_hash_bucket` queue, the thread is kept active and continuously checks the value of `thread_state`. This is equivalent to spinning on the flag and handing over to the CPU scheduler for managing sleeping. Notably, the change does not affect the semantics or the interface of futex. Meanwhile, user-space applications or libraries require no changes to use the new futex. In addition, the spinning on the flag in futex does not waste CPU cycles. Recall that if there are other runnable threads on the CPU,

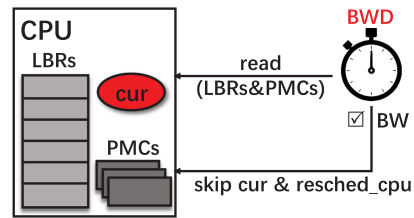


Fig. 7. Busy-waiting detection based on LBR.

threads in virtual blocking will not be scheduled and have no chance to waste CPU time spinning. On the other hand, if all threads are blocked, the spinning is not wasteful since no other valuable work can be scheduled.

Although the bucket queue preserves the original order of sleep and wakeup, we further modify the CPU scheduler to immediately schedule threads that are waking from virtual blocking, in a similar way the traditional wakeup process prioritizes those waking from real sleep. Note that VB only applies to sleep caused by futex and thus does not affect other sleep activities, e.g., blocking on I/O events. We also designed a mechanism to disable VB if threads are not oversubscribed. If the number of threads waiting on the bucket queue is smaller than the number of CPUs, i.e., all waiting threads can obtain a dedicated CPU when simultaneously waking up, VB is turned off.

4.3 Busy-Waiting Detection

As mentioned earlier, busy-waiting synchronization can cause cascading performance collapse among oversubscribed threads. Busy-waiting detection (BWD) attempts to detect useless busy-waiting operations and yield the busy-waiting threads so that CPU cycles can be used for valuable work. Similar to other existing detection mechanisms, BWD infers busy-waiting threads in the kernel without requiring to instrument application source code. Although there are various kinds of busy-waiting implementations, they all have one thing in common: a busy-waiting loop is just a small code segment that repeats many times. Specifically, 1) busy-waiting loop is usually backward conditional branches; 2) each iteration is very short and only takes a few cycles to execute; 3) during busy-waiting, branching is predictable with hundreds of thousands backward branching.

Although non-busy-waiting code segments may be similar to busy-waiting loops in some aspects, combining the three heuristics can reliably identify busy-waiting. BWD employs the last branch records (LBRs) and hardware performance counters (PMCs) as well as a high resolution timer to detect busy-waiting. The architecture of BWD is shown in Fig. 7. We configure a high resolution timer (`hrtimer`) [21] on each CPU to periodically check the LBRs and PMCs. If the readings of LBRs and PMCs indicate busy-waiting, the interrupt handler of the timer forces the currently running thread to relinquish the CPU. The LBRs record the from and to virtual addresses of the recently completed branches. Branches that are caused by function calls and returns are excluded from LBRs. This is to capture busy-waiting implementations that involve nested function calls. We further configure PMCs to record the number of TLB misses and L1 data cache misses.

TABLE 2
The Characteristics of Micro Benchmarks and NPB Benchmarks

Apps	Description	Memory(MB)	Locks blockspin
micro-b	pthread barriers test	7	✓
micro-s	spinlocks test	11	✓
is	integer sort	1062	✓
ep	embarrassingly parallel	12	✓
cg	conjugate gradient	893	✓
mg	multigrid	3419	✓
ft	3D fast fourier transform	5148	✓
sp	pentadiagonal solver	751	✓
bt	block tridiagonal solver	695	✓
ua	unstructured Adaptive mesh	483	✓
lu	lower-upper gauss-seidel solver	603	✓ ✓

BWD identifies busy-waiting if during the last timer interval (1) all branches recorded in LBRs (16 entries on our platform) were identical, backward branches, and 2) there were no data TLB misses or L1 data cache misses. The timer interval is carefully set to $100 \mu s$, the minimum interval that does not impose noticeable overhead. BWD requires that all the 16 entries be filled during an interval in order to identify a code segment as a busy-waiting loop. Recall that each iteration of a busy-waiting loop, which triggers a conditional branching, only takes a few cycles. It is almost certain that busy-waiting loops can fill all the 16 entries during the $100 \mu s$ interval. In comparison, non-busy-waiting loops can be, on average, at most $\frac{100 \mu s}{16} = 6.25 \mu s$ long to fill all the entries. Furthermore, BWD requires that there be no TLB misses or L1 data cache misses because the from and to addresses in busy-waiting loops are always cached in TLB and the data accesses in a tight busy-waiting loop should not miss any L1 data caches. On our Intel platform, TLB has only 1600 entries, and the L1 data cache is 32 KB. We conducted a comprehensive test on various programs and found that the combination of TLB and L1 data cache misses is a reliable metric for busy-waiting detection. Once the busy-waiting is detected, BWD deschedules the spinning thread and sets a skip flag on the thread. This ensures that the spinning thread will not be scheduled until other threads on the same CPU are scheduled at least once, which helps schedule critical threads sooner to avoid future spinning.

5 EVALUATION

Evaluation Setup. We used two micro-benchmarks (micro-b and micro-s) that are widely used for blocking and busy-

waiting synchronization primitives and nine real-world applications from NAS parallel benchmarks [13]. Table 2 lists the characteristics of those benchmarks. We evaluated on a 2-nodes, 16-CPU server with 128 GB memory and a 1 TB SATA hard drive. The OS used is Ubuntu 16.04 64 bit and the kernel version is 5.1.12. We test with five settings: 1) exactly one to one mapping of threads to CPUs in vanilla Linux (noted as #cpu-T (vanilla)); 2) dynamic threading of OpenMP (noted as #dyn-T (vanilla)); 3) threads equal to CPU request in vanilla Linux (noted as #req-T (vanilla)); 4) threads equal to CPU limit in vanilla Linux (noted as #lmt-T (vanilla)); 5) threads equal to CPU limit with specific optimizations enabled (noted as #lmt-T (VB), #lmt-T (BWD) or #lmt-T (ACS)).

5.1 Efficiency of Virtual Blocking

Micro-Benchmarks. First, we used the micro benchmark (micro-b) to test the efficiency of VB. We run the test set on a single CPU and created eight threads (an oversubscription ratio of 8) to repeatedly call pthread_barrier 10 k times, which stresses the threads trap into kernel futex to sleep and wakeup. The benchmark takes 1.98 seconds in vanilla Linux while only 1.25 seconds with VB enabled, indicating VB brings a 59.7% performance improvement. Fig. 8 shows the flame graph of kernel futex with and without VB. For the futex_wait, vanilla Linux takes much time to dequeue the threads from the CPU run queue to go to truly sleep, while the Linux with VB enabled only changes the state of the threads without wasting time for dequeue operations. For the futex_wake, vanilla Linux takes much time to wake up threads, while Linux with VB enabled only needs to update the state of newly woken threads to resume normal scheduling. Furthermore, VB can avoid many unnecessary migrations.

Macro-Benchmarks. Next, We started a container with a CPU request of 4 CPUs and a limit of 32 CPUs to run benchmarks from the NPB and vary the number of its accessible CPUs. As shown in the Fig. 9, dynamic threading of OpenMP brings up to 695% overhead as the collected statistics is inaccurate. Starting the number of threads equal to the CPU request in vanilla Linux performs well under full load, while cannot utilize more idle CPUs. However, starting the number of threads equal to the CPU limit performs well with enough CPUs, while brings up to 61% overhead in oversubscribed scenarios. For applications with fewer synchronization operations and memory accesses (e.g., ep), their performance is almost unaffected and even benefited. They do not suffer much of the inefficiency of synchronization and threads migrations. Other applications perform

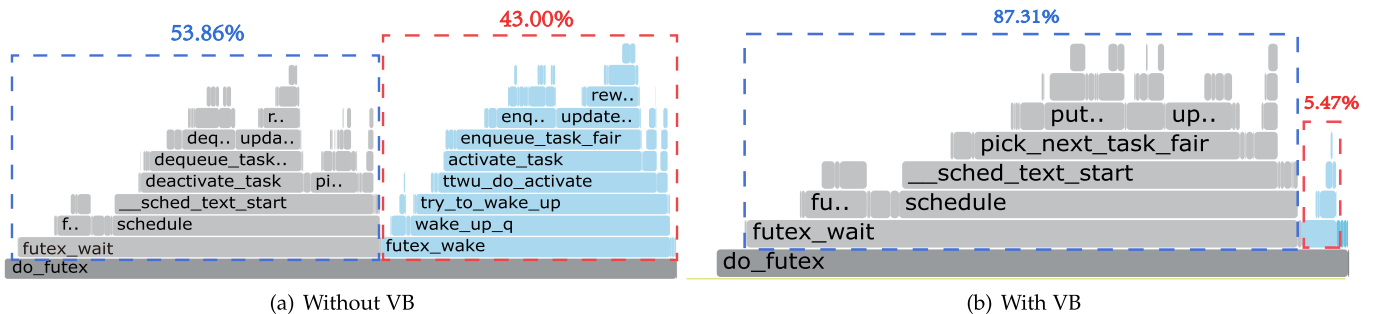


Fig. 8. The frame graph of kernel futex with and without VB.

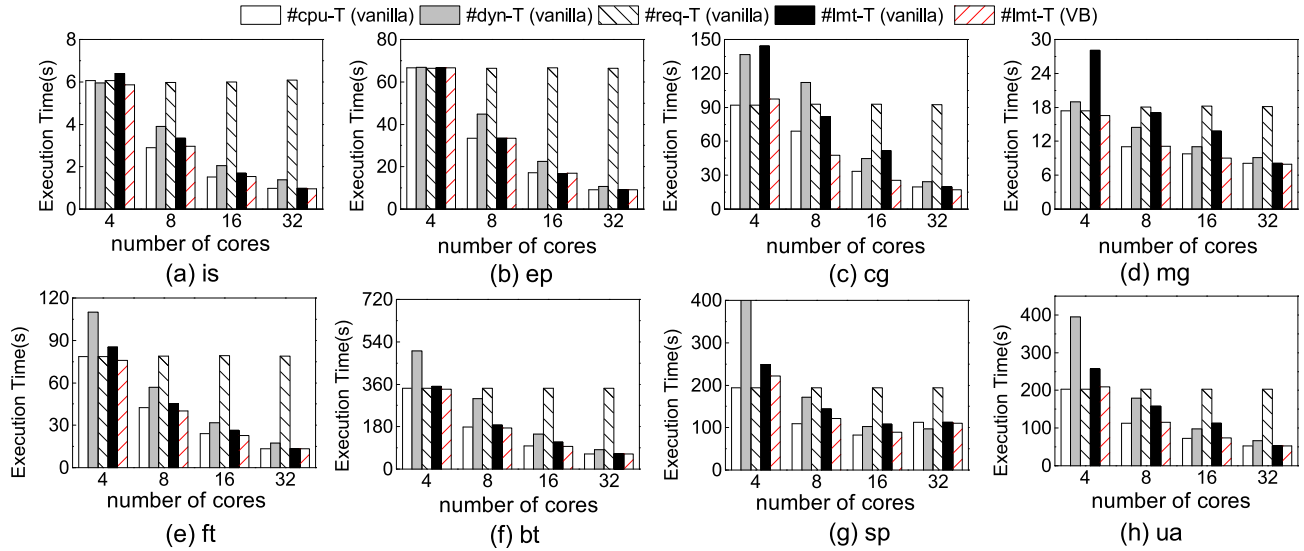


Fig. 9. The performance improvement with VB.

badly when the threads are more than the available CPUs. The inefficient synchronization leads to many threads migrations and causes much remote memory access. VB can avoid the performance loss caused by threads oversubscription, and even improve performance by up to 31%. The reason is that VB can greatly improve CPU utilization and reduce the number of thread migrations. The results prove that VB can keep blocking synchronization efficient, and burstable containers are encouraged to create enough threads to adapt to variable CPUs.

5.2 Efficiency of Busy-Waiting Detection

Micro-Benchmarks. To stress the busy-waiting detection (BWD), we proposed a microbenchmark (micro-s) with two threads, a worker thread, and an analysis thread. The worker thread performs the computing tasks withholding the spinlocks, and the analysis thread tries to acquire the spinlocks to analyze the results of the worker thread. Once the worker thread does not release the lock, the analysis thread stays busy waiting. We run the benchmark with different kinds of widely used spinlocks studied in [22] on a single CPU, and the baseline is acquired by removing all spinlocks. As shown in Fig. 10, BWD can detect busy-waiting in all kinds of busy-waiting implementations, achieve a similar performance as the baseline, and gain 48% performance improvement compared with vanilla Linux. Vanilla Linux without BWD results in 37% more instruction and 100% performance overhead as the analysis thread wastes time in meaningless busy-waiting instructions.

Macro-Benchmarks. We then started a container with a request of 4 CPUs and a limit of 32 CPUs to test if BWD work for customized busy-waiting algorithms in lu. As shown in Fig. 11, BWD can help lu achieve good performance. The dynamic threading algorithm is ineffective due to insufficient threads. Vanilla Linux performs poorly and brings up to 36x overhead as much time is wasted on executing busy-waiting instructions. The performance gain of BWD is up to 29.7x compared with vanilla Linux. Based on our tests, BWD achieved close to 100% true positive rates and up to 0.61% false positive rate.

5.3 Efficiency of Adaptive Container Scheduler

Micro-Benchmarks. To evaluate whether ACS can effectively place associative containers on adjacent CPUs, we containerize the shmempq benchmark [23] into a micro-service. Since its server and client contain a single process, we set their CPU request and limit to one CPU, and assign the same IPC namespace. Fig. 12 shows the average execution time is constant under different loads when ACS is enabled. However, it increases more than 200% when ACS is disabled. When there is only one group of server and client, the system contains many idle CPUs and migrates the containers between them, which leads to many remote memory accesses. When there are 16 groups of server and client, containers are less migrated between CPUs as there is no idle CPU to steal tasks. However, the service still perform badly with many remote memory access. In comparison, ACS can

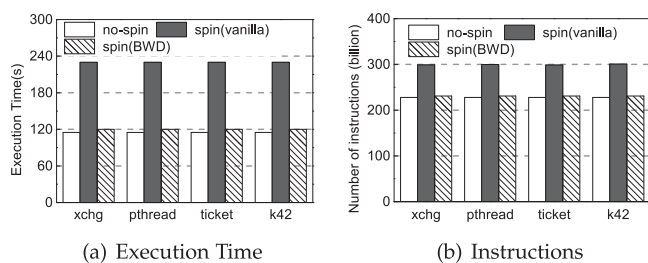


Fig. 10. The applicability of BWD to various spinlocks.

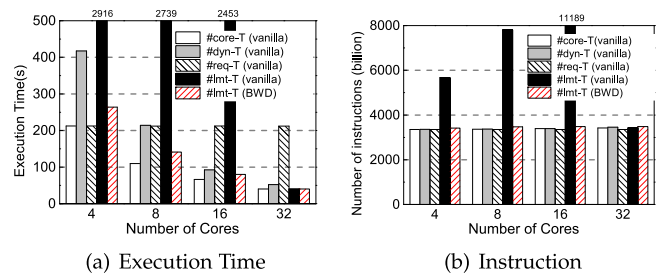


Fig. 11. The effectiveness of BWD in user-customized spinning.

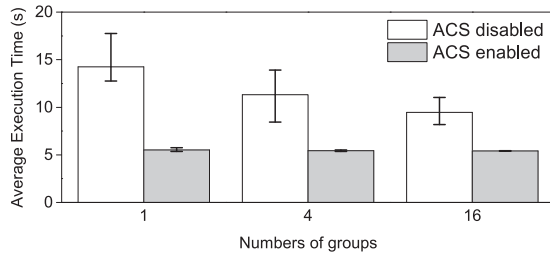


Fig. 12. The performance improvement with ACS.

detect associative containers based on their shared IPC namespace and always allocates adjacent CPUs, therefore avoiding remote memory access and keeping the service performance stable.

Macro-Benchmarks. Next, we test the applicability of ACS to containers with multi-thread applications under different loads. Each container is equipped with the request of 8 CPUs and the limit of 32 CPUs. As shown in Fig. 13, when only running one burstable container, it can utilize all 32 CPUs. Creating threads based on CPU requests cannot utilize idle CPUs and perform the worst. The dynamic threading also suffers a shortage of enough threads. Creating threads based on CPU limit can achieve better performance. In this case, ACS can extend the CPUs allocated to the container and brings slight overhead. However, when running four burstable containers, each can only obtain requested CPUs. As shown in Fig. 13a, creating threads based on CPU requests performs well as no threads oversubscription occurs. Creating threads based on CPU limits leads to the vanilla Linux unable to schedule and synchronize threads efficiently. The average execution time reaches up to 16 $\times$. The dynamic threading method still can not determine the best parallelism as it mistakes the global system statistics to make runtime decisions, brings up to 290% overhead. When creating 32 threads with only VB and BWD enabled, they can improve thread synchronization efficiency and performance of some applications (e.g., cg and lu). However, VB may reduce the efficiency of NUMA balancing and the performance of some applications (e.g., mg and ua). In contrast, when ACS is enabled, all applications perform well and even obtain up to 20% performance improvement compared with creating 8 threads. The above results indicate that ACS can help avoid scheduling overhead, and enabling VB and BWD can further mitigate the synchronization overhead. Once some containers finish, the remaining containers can utilize those idle CPUs.

6 RELATED WORK

The Suitable Degree of Parallelism. Existing approaches manage the DoP with different considerations [24]. Thread tailor [2] merges some threads based on communication patterns at runtime. AANT [25] and Aurora [26] adjust the DoP according to the behavior of program execution. Alvaro *et al.* [27] determine an optimized DoP through a machine learning model. Thread reinforcer [28] determines the appropriate DoP through runtime observations of multiple applications. Curtiset *et al.* [29] introduce a framework to adjust DoP based on offline analysis. The above work either requires application code changes [2], [25] or some form of offline analysis [28], [29]. In contrast, we suggest creating as

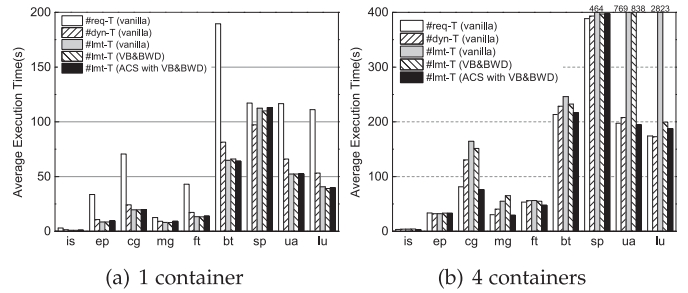


Fig. 13. The applicability of ACS to different loads.

many threads as the CPU limit for burstable containers to improve efficiency.

Management of Threads Scheduling. Many studies have proposed scheduling optimizations for performance improvement. For example, AutoNUMA [30] for Linux and some earlier research systems [31], [32], [33] implement locality-driven optimizations. Besides, other studies segregate the competing threads to different nodes to prevent contention. Zhuravlev *et al.* [34] proposed a scheduling algorithm to distribute threads among CPUs based on the last-level cache misses. Merkel *et al.* [35] introduce a co-scheduling policy to avoid resource contention. However, the above studies lack container awareness, resulting in randomly scheduling threads on the same containers. Other approaches [9], [36] determine the appropriate placement of threads based on prediction, which is not suitable for complex container environments. Instead, we propose a user-level container scheduler for container management that assigns dedicated CPUs to containers by considering multiple dimensions.

Thread Synchronization. Existing work [37] studies the thread inefficiencies caused by lock contention. Dice *et al.* [38] introduces a compact NUMA-aware lock, and Kashyap *et al.* [17] proposed a shuffling lock to improve synchronization. Spinlocks are considered more harmful under high contention [39]. Li *et al.* [40] implemented a hardware-based simulator to detect spinning. Chakraborty *et al.* [41] detected spinning for vCPUs. Our findings suggest that L1 TLB misses and data cache misses together with branch addresses are reliable methods for busy-waiting detection. BWD utilizes the features (LBR and PMU) of modern CPUs and is useful for various spin implementations.

7 CONCLUSION

With the development of cloud-native, adapting burstable containers to variable CPU resources is essential for improving the application performance. In this paper, we demonstrate that the central overheads of managing burstable containers come from threads scheduling and synchronization. To that end, we proposed a user-level container scheduler and two synchronization mechanisms that can manage burstable containers with excessive threads effectively. Experimental results of micro- and macro-benchmarks show that our mechanisms can make a significant improvement in the management of burstable containers.

ACKNOWLEDGMENTS

The authors would like to thank the anonymous reviewers for their helpful feedback.

REFERENCES

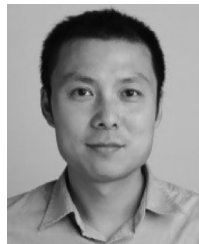
- [1] W. Chen, A. Pi, S. Wang, and X. Zhou, "Pufferfish: Container-driven elastic memory management for data-intensive applications," in *Proc. ACM Symp. Cloud Comput.*, 2019, pp. 259–271.
- [2] J. Lee, H. Wu, M. Ravichandran, and N. Clark, "Thread tailor: Dynamically weaving threads together for efficient, adaptive parallel applications," in *Proc. Annu. Int. Symp. Comput. Architect.*, 2010, pp. 270–279.
- [3] H. Qin, Q. Li, J. Speiser, P. Kraft, and J. Ousterhout, "Arachne: Core-aware thread management," in *Proc. USENIX Symp. Operating Syst. Des. Implementation*, 2018, pp. 145–160.
- [4] Y. Cho, C. A. Celis Guzman, and B. Egger, "Maximizing system utilization via parallelism management for co-located parallel applications," in *Proc. Int. Conf. Parallel Architect. Compilation Techn.*, 2018, pp. 1–14.
- [5] L. Dagum and R. Menon, "OpenMP: An industry-standard API for shared-memory programming," *IEEE Comput. Sci. Eng.*, vol. 5, no. 1, pp. 46–55, First Quarter 1998.
- [6] H. Huang, J. Rao, S. Wu, H. Jin, K. Suo, and X. Wu, "Adaptive resource views for containers," in *Proc. Int. Symp. High- Perform. Parallel Distrib. Comput.*, 2019, pp. 243–254.
- [7] E. H. Molina da Cruz, M. A. Zanata Alves, A. Carissimi, P. O. A. Navaux, C. P. Ribeiro, and J. -F. Mehaut, "Using memory access traces to map threads and data on hierarchical multi-core platforms," in *Proc. IEEE Int. Symp. Parallel Distrib. Process. Workshops Phd Forum*, 2011, pp. 551–558.
- [8] N. Denoyelle, B. Goglin, E. Jeannot, and T. Ropars, "Data and thread placement in numa architectures: A statistical learning approach," in *Proc. Int. Conf. Parallel Process.*, 2019, pp. 1–10.
- [9] D. Goodman, G. Varisteas, and T. Harris, "Pandia: Comprehensive contention-sensitive thread placement," in *Proc. Eur. Conf. Comput. Syst.*, 2017, pp. 254–269.
- [10] T. Harris, M. Maas, and V. J. Marathe, "Callisto: Co-scheduling parallel runtime systems," in *Proc. Eur. Conf. Comput. Syst.*, 2014, pp. 1–14.
- [11] M. Diener, E. H. M. Cruz, P. O. A. Navaux, A. Busse, and H. -U. HeiB, "kMAF: Automatic kernel-level management of thread and data affinity," in *Proc. 23rd Int. Conf. Parallel Architect. Compilation*, 2014, pp. 277–288.
- [12] B. Lepers, V. Quéma, and A. Fedorova, "Thread and memory placement on NUMA systems: Asymmetry matters," in *Proc. USENIX Annu. Tech. Conf.*, 2015, pp. 277–289.
- [13] NPB. *NAS Parallel Benchmarks*, 2019. [Online]. Available: <https://www.nas.nasa.gov/>
- [14] *Alibaba Cluster Trace Prog.*, 2018. [Online]. Available: <https://github.com/alibaba/clusterdata>
- [15] W. Heirman, T. E. Carlson, K. Van Craeynest, I. Hur, A. Jaleel, and L. Eeckhout, "Undersubscribed threading on clustered cache architectures," in *Proc. IEEE 20th Int. Symp. High Perform. Comput. Architect.*, 2014, pp. 678–689.
- [16] K. Kumar Pusukuri, R. Gupta, and L. N. Bhuyan, "ADAPT: A framework for coscheduling multithreaded programs," *ACM Trans. Archit. Code Optim.*, vol. 9, no. 4, pp. 1–24, 2013.
- [17] S. Kashyap, I. Calciu, X. Cheng, C. Min, and T. Kim, "Scalable and practical locking with shuffling," in *Proc. ACM Symp. Operating Syst. Princ.*, 2019, pp. 586–599.
- [18] D. Dice, "Malthusian locks," in *Proc. Eur. Conf. Comput. Syst.*, 2017, pp. 314–327.
- [19] G. Chrysos, "Intel Xeon Phi coprocessor-the architecture," *Intel Whitepaper*, vol. 176, no. 2014, pp. 43–50, 2014.
- [20] S. Schildermans, J. Shan, K. Aerts, J. Jackrel, and X. Ding, "Virtualization overhead of multithreading in X86 state-of-the-art & remaining challenges," *IEEE Trans. Parallel Distrib. Syst.*, vol. 132, no. 10, pp. 2557–2570, 2021.
- [21] T. Gleixner and D. Niehaus, "Hrtimers and beyond: Transforming the linux time subsystems," in *Proc. Linux Symp.*, 2006, pp. 333–346.
- [22] Spinlocks and Read-Write Locks, 2019. [Online]. Available: <http://locklessinc.com/articles/locks/>
- [23] Shmemq, 2022. [Online]. Available: <https://github.com/goldshn/shmemq-blog>
- [24] C. Iancu, S. Hofmeyr, F. Blagojević, and Y. Zheng, "Oversubscription on multicore processors," in *Proc. IEEE Int. Symp. Parallel Distrib. Process.*, 2010, pp. 1–11.
- [25] A. F. Lorenzon, J. D. Souza, and A. C. S. Beck, "LAANT: A library to automatically optimize EDP for OpenMP applications," in *Proc. Des., Automat. Test Europe Conf. Exhib.*, 2017, pp. 1229–1232.
- [26] A. F. Lorenzon, C. C. De Oliveira, J. D. Souza, and A. C. S. Beck, Aurora: Seamless optimization of OpenMP applications, *IEEE Trans. Parallel Distrib. Syst.*, vol. 30, no. 5, pp. 1007–1021, May 2019.
- [27] Á. B. Hernández, M. S.S. PerezGupta, and V. Muntés-Mulero, "Using machine learning to optimize parallelism in Big Data applications," *Future Gener. Comput. Syst.*, vol. 86, pp. 1076–1092, 2018.
- [28] K. K. Pusukuri, R. Gupta, and L. N. Bhuyan, "Thread reinforcer: Dynamically determining number of threads via os level monitoring," in *Proc. IEEE Int. Symp. Workload Characterization*, 2011, pp. 116–125.
- [29] M. Curtis-Maury, J. Dzierwa, C. D. Antonopoulos, and D. S. Nikolopoulos, "Online power-performance adaptation of multithreaded programs using hardware event-based prediction," in *Proc. 20th Annu. Int. Conf. Supercomput.*, 2006, pp. 157–166.
- [30] AutoNUMA, 2021. [Online]. Available: <https://lwn.net/Articles/488709/>
- [31] T. Brecht, "On the importance of parallel application placement in numa multiprocessors," in *Proc. Symp. Experiences Distrib. Multiprocessor Syst.*, 1993, pp. 1–18.
- [32] B. Verghese, S. Devine, A. Gupta, and M. Rosenblum, "Operating system support for improving data locality on cc-numa compute servers," *ACM SIGOPS Operating Syst. Rev.*, vol. 30, no. 5, pp. 279–289, 1996.
- [33] R. Lachaize, B. Lepers, and V. Quéma, "MemProf: A memory profiler for NUMA multicore systems," in *Proc. USENIX Annu. Tech. Conf.*, 2012, pp. 53–64.
- [34] S. Zhuravlev, S. Blagodurov, and A. Fedorova, "Addressing shared resource contention in multicore processors via scheduling," *ACM Sigplan Notices*, vol. 45, no. 3, pp. 129–142, 2010.
- [35] A. Merkel, J. Stoess, and F. Belloso, "Resource-conscious scheduling for energy efficiency on multicore processors," in *Proc. 5th Eur. Conf. Comput. Syst.*, 2010, pp. 153–166.
- [36] J. Funston et al., "Placement of virtual containers on numa systems: A practical and comprehensive model," in *Proc. USENIX Annu. Tech. Conf.*, 2018, pp. 281–294.
- [37] H. Guiroux, R. Lachaize, and V. Quéma, "Multicore locks: The case is not closed yet," in *Proc. USENIX Annu. Tech. Conf.*, 2016, pp. 649–662.
- [38] D. Dice and A. Kogan, "Compact NUMA-aware locks," in *Proc. 14th EuroSys Conf.*, 2019, pp. 1–15.
- [39] W. Xiong, S. Park, J. Zhang, Y. Zhou, and Z. Ma, "Ad hoc synchronization considered harmful," in *Proc. USENIX Symp. Operating Syst. Des. Implementation*, 2010, pp. 163–176.
- [40] T. Li, A. R. Lebeck, and D. J. Sorin, "Spin detection hardware for improved management of multithreaded systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 17, no. 6, pp. 508–521, Jun. 2006.
- [41] K. Chakraborty, P. M. Wells, and G. S. Sohi, "Supporting over-committed virtual machines through hardware spin detection," *IEEE Trans. Parallel Distrib. Syst.*, vol. 23, no. 2, pp. 353–366, Feb. 2012.



Hang Huang received the BS degree from the Huazhong University of Science and Technology (HUST), in 2016. He is currently working toward the PhD degree with Service Computing Technology and System Lab (SCTS), Cluster and Grid Computing Lab (CGCL), and Big Data Technology and System Lab (BDTS), Huazhong University of Science and Technology (HUST), China. His current research interests include container technology and kernel scheduling.



Yuqing Zhao received the bachelor's degree in computer science and technology from Central South University (CSU), in 2019. She is currently working toward the master's degree with Service Computing Technology and System Lab (SCTS), Cluster and Grid Computing Lab (CGCL), and Big Data Technology and System Lab (BDTS), Huazhong University of Science and Technology (HUST), China. Her current research interests include cloud computing and virtualization.



Jia Rao (Member, IEEE) received the BS and MS degrees in computer science from Wuhan University, in 2004 and 2006, respectively, and the PhD degree from Wayne State University, in 2011. He is currently an associate professor of computer science with the University of Texas, Arlington. His research interests include the areas of distributed systems, resource auto-configuration, machine learning, and CPU scheduling on emerging multi-core systems.



Duoqiang Wang is currently an associate professor with the School of Computer Science and Technology, Huazhong University of Science and Technology. He has published many articles in journals and conferences. His research interests include cloud computing, high-performance computing, and parallel algorithms.



Song Wu (Member, IEEE) received the PhD degree from Huazhong University of Science and Technology (HUST), in 2003. He is currently a professor of computer science with the Huazhong University of Science and Technology, China. He currently serves as the vice dean with the School of Computer Science and Technology and the vice head with Service Computing Technology and System Lab (SCTS), the Cluster and Grid Computing Lab (CGCL), and Big Data Technology and System Lab (BDTS), Huazhong University of Science and Technology. His current research interests include cloud resource scheduling and system virtualization.

of Science and Technology. His current research interests include cloud resource scheduling and system virtualization.



Hai Jin (Fellow, IEEE) received the PhD degree in computer engineering from Huazhong University of Science and Technology (HUST), in 1994. He is currently a Cheung Kung Scholars chair professor of computer science and engineering with the Huazhong University of Science and Technology, China. In 1996, he was awarded a German Academic Exchange Service fellowship to visit the Technical University of Chemnitz in Germany. He worked with The University of Hong Kong between 1998 and 2000, and as a visiting

scholar with the University of Southern California between 1999 and 2000. He has coauthored 22 books and published more than 800 research papers. His research interests include computer architecture, virtualization technology, cloud computing, peer-to-peer computing, and network storage. He is a member of the ACM.



Suo Kun (Member, IEEE) received the BS degree in software engineering from the Nanjing University, China, in 2012, and PhD degree from the University of Texas, Arlington, in 2019. He is currently an assistant professor with the Department of Computer Science, Kennesaw State University. His research interests include the areas of cloud computing, virtualization, operating systems, Java virtual machines, and software defined network.



Lisong Pan is working toward the undergraduate degree with the Huazhong University of Science and Technology. He is currently doing an internship with Service Computing Technology and System Lab (SCTS), Cluster and Grid Computing Lab (CGCL), and Big Data Technology and System Lab (BDTS). His research interests mainly include virtualization and cloud computing.

► **For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/csdl.**